Audit technique — Nhill (base 0.1.0, version cible 2.0.0)

> **Date :** 26 octobre 2026 · **Périmètre :** dépôt complet livré (`nhill-0.1.0-src.tar.gz`) et livrables 0.1.0 (`nhill-release/`) · **Méthode :** lecture intégrale de l'arborescence, mesures automatiques (lignes, routes, tables, tests), exécution des tests réalisables, exécution des binaires 0.1.0 livrés sous Xvfb. **Limite majeure de l'environnement d'audit :** seul GitHub est joignable ; les dépôts Ubuntu/Debian, Qt, PyPI, npm et Google sont bloqués, et la politique de sécurité de la session a refusé la compilation des dépendances depuis leurs sources. Le projet complet (serveur + applications Qt) **n'a donc pas pu être recompilé ici**. Ce qui a été exécuté est indiqué explicitement.

1. Synthèse

Nhill est une suite de gestion commerciale multi-établissements, écrite en **C++17**, composée d'un serveur HTTP/JSON et de trois applications **Qt 6 Quick (QML)** partageant une même bibliothèque. La qualité d'ensemble est **bonne et au-dessus de la moyenne** pour un produit à ce stade : séparation nette client/serveur, calculs monétaires exacts en unités mineures, isolation des établissements par *row-level security* PostgreSQL forcée, migrations versionnées avec empreintes, journalisation d'audit, tests d'intégration réels contre un PostgreSQL jetable.

Les faiblesses principales ne sont pas dans la logique métier mais dans :

1. **la chaîne de livraison** (Windows et Android jamais compilés ; paquets Debian 13 construits sur Ubuntu avec un runtime Qt embarqué figé, sans mises à jour de sécurité) ;
2. **l'ergonomie de présentation** (tailles de fenêtres fixes, tableaux tronqués, contrastes insuffisants — voir l'audit UI/UX) ;
3. **des dépendances embarquées vulnérables** (cpp-httplib 0.14, Qt 6.4.2 figé — voir l'audit de sécurité) ;
4. **l'absence de parcours de récupération de compte** (pas d'envoi d'e-mails).

Domaine	Appréciation	Points saillants
Architecture	Solide	Client léger, serveur autoritaire, une base de code pour 3 applications
Logique métier	Solide, testée	Montants exacts, taxes, numérotation transactionnelle, immuabilité des documents émis
Données	Solide	10 migrations, 64 tables, RLS forcée sur 30 tables métier
Tests	Bons	4 suites (common, server, setup, client) ; 0.1.0 : 119 tests, 1 ignoré
Construction	Fragile	Seul Linux construit ; dépendance à des scripts hors dépôt (corrigé en 2.0.0)
Distribution	Incomplète (0.1.0)	Windows/Android non construits ; Debian 13 avec runtime embarqué — corrigé en 2.0.0 : installateurs Windows, APK, paquets Debian 13 natifs construits en CI (rapport de validation)
Observabilité	Minimale	`/health`, journal systemd, « Erreurs techniques » de la console

2. Inventaire

2.1 Langages, normes et volumes

Élément	Valeur mesurée
Langage	C++17 (`CMAKE_CXX_STANDARD 17`, extensions désactivées)
Serveur	10 765 lignes C++ (`server/src`), **182 routes** HTTP enregistrées
Client (C++)	1 500 lignes (`client/src/core`, 3 `main.cpp`)
Client (QML)	8 955 lignes, 95 fichiers, 4 modules (`Nhill.Ui`, `Nhill.Business`, `Nhill.Console`, `Nhill.Pos`)
Bibliothèque commune	`common/` : montants, codes d'établissement, validations (≈ 690 lignes)
Base de données	10 migrations SQL (1 709 lignes), **64 tables**, schéma dédié `nhill`
Traductions	`client/i18n/nhill_fr.ts` / `nhill_en.ts` (≈ 1 900 messages), `server/i18n/{fr,en}.json`

2.2 Cadriciels et bibliothèques

Composant	Utilisé par	Version de référence (0.1.0)	Remarque
Qt 6 (Core, Gui, Network, Qml, Quick, QuickControls2, Svg, LinguistTools, Test)	Client	6.4.2 (Ubuntu 24.04)	Style « Basic », composants maison `Fb*`
QtKeychain (Qt 6)	Client	0.14 (facultatif)	Jetons de rafraîchissement dans le trousseau du système ; pas de repli en clair
libpqxx	Serveur	7.8	Accès PostgreSQL
PostgreSQL	Serveur	15+ (tests : 16 ; production : Supabase 17.6)	RLS forcée, rôles dédiés
cpp-httplib	Serveur, tests client	0.14	**Vulnérable** (CVE-2025-46728, CVE-2025-53629), voir audit sécurité
nlohmann/json	Serveur	≥ 3.10	
libsodium	Serveur	1.0.18	Argon2id, secretbox, HMAC
OpenSSL	Serveur	3.x	Hachages, TLS sortant (prestataires)
libharu (libhpdf)	Serveur	2.3.0	PDF des factures, tickets, rapports
libpng	Serveur	1.6	Logos
GoogleTest	Tests	1.14	

2.3 Modules et points d'entrée

```
common/            nhill_common (statique)   montants, taxes, conversions, codes, validations
server/src/        nhill_server_core + nhill-server
  main.cpp           sous-commandes : serve, migrate, setup (mode installation)
  http/app.*         serveur HTTP, contexte, permissions par rôle, erreurs JSON
  services/*.cpp     auth, établissements, membres, abonnements, prestataires, console,
                     assistance, POS, catalogue, tiers, stock, ventes, achats, paiements,
                     dépenses, rapports, activité, administration
  pdf/, jobs/, setup.cpp, migrate.cpp, util/crypto.cpp
client/src/        nhill_client_core (ApiClient, Session, Formatter, Validator, AppContext)
  business/ pos/ console/ main.cpp   trois exécutables
client/qml/Nhill/  Ui (composants), Business, Console, Pos (écrans)
db/migrations/     schéma versionné 0001–0010 (empreintes SHA-256 vérifiées)
tests/             common/, server/ (API + PostgreSQL jetable, installation), client/
deploy/            systemd, Caddy (TLS), sauvegarde/restauration chiffrées
```

2.4 Système de construction

- **CMake ≥ 3.21 + Ninja**, options `NHILL_BUILD_SERVER/CLIENT/TESTS`, `NHILL_WARNINGS_AS_ERRORS`. Avertissements stricts (`-Wall -Wextra`, `/W4`).
- Modules QML compilés par `qt_add_qml_module` (qmlcachegen) ; ressources (polices IBM Plex, logos SVG) embarquées ; traductions compilées par `qt_add_lrelease`.
- Paquets : CPack (`cmake/NhillPackaging.cmake`) et, dans les livrables 0.1.0, des scripts `build-deb.sh` / `build-deb-debian13.sh` **situés hors du dépôt source** (corrigé : intégrés dans `packaging/` en 2.0.0).
- Windows : script PowerShell + NSIS/Inno Setup **jamais exécutés** ; Android : script `build-android.sh` **jamais exécuté**.

2.5 Dépendances système (exécution)

- **Applications :** Qt 6 et ses modules QML, greffon de plate-forme xcb (X11) ou Wayland, OpenGL, fontconfig, D-Bus/Secret Service (trousseau).
- **Serveur :** libpq5, libsodium23, libssl3, libhpdf, libpng, zlib, systemd, utilisateur système `nhill`.
- Vérification réalisée : les binaires 0.1.0 livrés démarrent ici avec le runtime Qt embarqué ; il manque sur cette machine `libOpenGL.so.0` et `libb2.so.1` (fournis par `libopengl0` et `libb2-1` sur une machine normale ; remplacés ici par des cales de test).

2.6 Base de données

- PostgreSQL 15+ ; schéma `nhill` fermé à `PUBLIC`, `anon`, `authenticated`, `service_role`.
- Rôles : `nhill_owner` (migrations), `nhill_app` (application, `NOBYPASSRLS`, ne possède aucun objet), `nhill_backup` (sauvegardes).
- RLS forcée sur 30 tables métier (`establishment_id = nh_current_establishment()`), clés étrangères composites empêchant les références inter-établissements.
- 18 tables de plate-forme sans RLS par conception (sessions, abonnements, journal…), protégées par les requêtes du serveur (revue documentée dans `docs/archive/securite.md`).
- Migrations immuables : une migration modifiée après application est refusée.

2.7 Configuration

- Serveur : `/etc/nhill/server.env` (0600, refusé s'il est lisible par d'autres), variables `NH_*` (base, écoute, secrets, Argon2, durée des jetons, tâches de fond). Exemple sans secret : `config/server.env.example`.
- Clients : préférences Qt (`QSettings`) — adresse du serveur, langue, thème. **Aucune donnée métier n'est stockée localement**, aucun secret de base de données dans les applications.
- Mode installation : un serveur sans base démarre en mode `setup` et affiche un jeton à usage unique dans son journal ; Nhill Console pilote l'installation.

2.8 Authentification, autorisations, sessions

- Mots de passe Argon2id (libsodium, 3 passes, 64 Mio), comparaison en temps constant, hachage factice pour les e-mails inconnus.
- Jetons opaques (32 octets) stockés sous forme de SHA-256 ; accès 15 min, rafraîchissement 30 jours avec rotation et détection de réutilisation (révocation de la famille).
- Rôles d'établissement : `owner`, `admin`, `manager`, `sales`, `accountant`, `viewer`, `cash_manager`, `cashier` ; permissions vérifiées côté serveur (`App::roleHas`).
- Console : comptes distincts, mot de passe + **TOTP obligatoire**, rôles `superadmin`, `billing`, `support`, `auditor`, jeton en mémoire uniquement.
- Limitation des tentatives (connexion, codes, invitations, console, POS…) en base (`auth_throttle`), donc valable sur plusieurs processus.
- **Lacune :** aucun parcours « mot de passe oublié » (pas de transport e-mail).

2.9 Erreurs et journaux

- Erreurs API JSON normalisées (`code`, `message`, `details`, `request_id`) ; correspondance des erreurs SQL (RLS → 404 sans fuite, contraintes → codes stables).
- Journaux sur la sortie standard (journald) ; e-mails « transport log » masqués.
- Les applications envoient leurs erreurs QML au serveur (données personnelles masquées) ; la console les regroupe par empreinte.
- **Constat :** ces rapports ont révélé deux erreurs réelles en 0.1.0 (TypeError dans `PosSession.qml` et `total` de `null` dans Nhill Business) ; la première est corrigée en 2.0.0.

2.10 Réseau, ressources, fonctionnement en ligne

- HTTP/JSON sur `/api/v1`, TLS terminé par Caddy (HSTS, en-têtes de sécurité, limite de corps 6 Mo) ; le serveur n'écoute que sur 127.0.0.1.
- **Fonctionnement exclusivement en ligne** : pas de mode hors ligne (choix documenté : stock, numérotation et droits arbitrés par le serveur). Les ventes POS sont protégées contre les doublons par clé d'idempotence. Contrainte forte pour le marché cible (voir étude de marché).

3. État initial de la compilation et des tests (référence)

Vérification	Résultat	Statut
Tests métier `tests/common` (20 tests en 2.0.0, 18 en 0.1.0) compilés avec g++ 13 et un exécuteur compatible GoogleTest (`scripts/test-common-standalone.sh`)	0.1.0 : **18/18 réussis** ; 2.0.0 : **20/20 réussis**	Réalisé et testé

Vérification	Résultat	Statut
Contrôle statique QML (`scripts/qa/qml-check.py` : équilibre des délimiteurs, analyse JavaScript de 4 201 liaisons/fonctions par Node.js, modules, traductions)	0.1.0 : 0 erreur ; 2.0.0 : 0 erreur, 0 chaîne non traduite	Réalisé et testé
Démarrage des binaires 0.1.0 livrés (Xvfb 1366×768)	Nhill POS, Business, Console démarrent, écran de connexion affiché	Réalisé et testé
Compilation complète (serveur + clients) et `ctest`	**Non exécutable ici** (dépendances indisponibles)	Bloqué — exécuté par la CI fournie (`.github/workflows/ci.yml`)
Rapport de tests 0.1.0 du projet	18 + 82 (1 ignoré) + 6 + 13 réussis	Repris de `docs/archive/rapport-tests.md`

4. Défauts et dettes techniques identifiés

#	Constat	Gravité	Traitement 2.0.0
T1	Scripts de paquets de la livraison 0.1.0 hors du dépôt source	Moyenne	Intégrés dans `packaging/` ; versions lues dans `CMakeLists.txt`
T2	Tests serveur liés à `/usr/lib/postgresql/16` (échec sur Debian 13, PostgreSQL 17)	Moyenne	Détection automatique de la version (`pgBinDir`)
T3	Windows et Android jamais compilés	Haute (distribution)	Workflows CI reproductibles ; à exécuter (voir guide de compilation)
T4	Paquets Debian 13 avec Qt 6.4.2/ICU/libjpeg embarqués, figés	Haute (sécurité)	Construction native Debian 13 (Qt 6.8 du système) par la CI ; variante embarquée déconseillée
T5	cpp-httplib 0.14 vulnérable (DoS par corps « chunked »)	Haute	Atténué par Caddy (`request_body max_size`) ; mise à niveau ≥ 0.23 recommandée (non faite : non testable ici)
T6	Fenêtres de taille fixe (1280×800/820, 1320×840)	Haute (UX)	Taille adaptée à l'écran, centrage, plein écran sur petits écrans
T7	`X-Forwarded-For` lu à gauche (falsifiable derrière un proxy qui ajoute)	Faible (Caddy fourni remplace l'en-tête)	Lecture de l'entrée la plus à droite
T8	Actions de caisse, d'assistance et de console sans libellé dans le journal d'activité	Moyenne (UX/traçabilité)	51 libellés FR/EN ajoutés ; identifiants techniques masqués
T9	Aucun parcours de récupération de compte	Haute (exploitation)	Non corrigé (exige l'envoi d'e-mails) — conception proposée, à planifier
T10	Pas de mode hors ligne	Moyenne (marché)	Hors périmètre 2.0.0 (changement d'architecture)
T11	Pas de facturation normalisée DGI (SFE/DEF)	Haute (conformité RDC)	Hors périmètre ; feuille de route d'homologation
T12	Duplication `scripts/build-*.ps1/sh` et livrables	Faible	Point d'entrée unique dans `scripts/`, implémentation dans `packaging/`
T13	Champs QML dont la largeur dépend du texte saisi (grilles déséquilibrées)	Moyenne (UX)	Largeur préférée stable des champs
T14	Course dans le dialogue de ticket POS (`full` nul)	Moyenne	Garde ajoutée
T15	Aucun outil de contrôle QML sans Qt	Faible	`scripts/qa/qml-check.py`

5. Modifications 2.0.0 susceptibles d'affecter le comportement métier

Conformément à la règle de préservation de la logique métier, chaque changement est listé avec sa justification et son test :

Changement	Justification	Effet métier	Test
Longueur minimale des mots de passe 10 → **8** (14 → 8 pour la console)	Exigence du propriétaire du produit	Règle d'authentification assouplie ; refus des mots de passe courants, répétitifs ou égaux à l'e-mail **conservé** ; TOTP console **conservé**	`Validation.PasswordMinimumIsEightCharacters`, `Validation.PasswordBlocklistCoversEightCharacterFavourites` (exécutés), `Auth.PasswordMinimumIsEightCharactersOnEveryPath`, `Admin.ConsoleFirstAccessAcceptsEightCharactersWithTotp` (CI)
Liste de mots de passe courants étendue (8–9 caractères)	Compenser la longueur réduite	Quelques mots de passe faibles refusés	Tests ci-dessus
`X-Forwarded-For` : entrée de droite	Correctif de sécurité	Limites par IP fiables quel que soit le proxy	Logique testée isolément (6 cas)
Résumé du journal d'activité sans identifiants techniques	Lisibilité, minimisation	Aucun (données brutes inchangées en base et dans l'API)	Revue de code
Contrôle du comptage des espèces avant clôture POS	Le serveur l'exige déjà (`cash_count_required`)	Aucun (message plus tôt)	Revue de code

Aucune modification des calculs, du modèle de données, des migrations, des règles de facturation et d'abonnement, des permissions ni des formats d'API.

6. Recommandations priorisées

1. **Exécuter la CI fournie** (compilation, 4 suites de tests, paquets Ubuntu 24.04 et Debian 13 natifs, Windows, Android) et publier les artefacts — c'est la seule étape qui transforme le code 2.0.0 en livrables vérifiés.
2. Mettre à niveau cpp-httplib (≥ 0.23, en-tête unique vendu dans le dépôt) et abandonner le runtime Qt embarqué pour Debian 13.
3. Implémenter l'envoi d'e-mails (SMTP) puis la réinitialisation de mot de passe (lien à usage unique, durée courte, règle des 8 caractères).
4. Engager l'homologation SFE/DEF auprès de la DGI avant de cibler les assujettis à la TVA.
5. Étudier un mode dégradé hors ligne pour Nhill POS (file d'attente locale chiffrée, numérotation provisoire) — changement d'architecture à concevoir avec soin.
6. Ajouter une supervision (métriques, alertes de sauvegarde) avant la mise en production.